

CSCE 658 RANDOMIZED ALGORITHMS · MODULE 07 · LECTURE NOTES

The Probabilistic Method

Proving things exist by picking one at random.

MODULE OUTCOMES

- Apply the basic probabilistic method to an existence proof.
- Apply the expectation argument and the deletion method.
- State and apply the Lovász local lemma.
- Explain the alteration and second-moment variants.
- Explain how a probabilistic existence proof becomes an algorithm.

1 The basic method

THE METHOD: to prove that an object with property P exists, define a probability distribution over candidate objects and show that $P(\text{a random candidate has property } P) > 0$.

Then at least one candidate must have property P . Done.
No construction, no explicit object -- just existence.

EXAMPLE: every graph has a cut containing at least half its edges.

Put each vertex independently on side A or side B, by a fair coin flip.

Each edge crosses the cut with probability exactly $1/2$.
So $E[\text{number of edges crossing}] = m/2$, by linearity of expectation.

An average is attained or exceeded by some outcome.
So SOME assignment cuts at least $m/2$ edges. Done.

THAT IS A COMPLETE PROOF of a non-trivial combinatorial fact, in five lines, with no construction whatsoever.

AND IT IS CONSTRUCTIVE ANYWAY: the random assignment achieves $m/2$ in expectation, so it is a $1/2$ -approximation algorithm for MAX-CUT -- which is CSCE 669 Module 10, and is exactly the baseline that the semidefinite programming relaxation improves on (to 0.878).

LINEARITY OF EXPECTATION DOES MOST OF THE WORK

$E[X + Y] = E[X] + E[Y]$, regardless of any dependence between X and Y . No independence assumption at all — which is why it applies in situations where Chernoff bounds do not (Module 02 §2's requirement).

So a count of structures is a sum of indicator variables, and its expectation is the sum of their probabilities — computable even when the indicators are heavily dependent, which is the usual situation in combinatorics.

Which is the whole technique: express the quantity of interest as a sum, take expectations term by term, and conclude that some outcome achieves at least the average.

And it is the most underused tool in applied probability — people reach for an independence assumption when linearity would have sufficed, and linearity is free and unconditional. **Whenever you find yourself worrying about dependence, check whether you only need the mean.**

2 The variants

Variant	When to use it	How it works
Expectation argument	You want a quantitative bound, not merely existence.	Some outcome meets or exceeds the average — §1's MAX-CUT argument.
Alteration	The random object is nearly right but has some violations.	Fix the violations by hand and bound how much the fixing costs — used for Ramsey-type and independent-set bounds.
Deletion	The expected number of bad structures is positive but small.	Build the object anyway, delete the bad structures, and bound how much of the object is lost — see the note.
Second moment	You need concentration rather than just the mean.	Compute the variance and apply Chebyshev (Module 06 §2's second moment method).
Lovász local lemma	Many bad events, each unlikely, with only local dependence.	§3 — the strongest tool in the module.

The deletion method is the one to reach for when a first attempt gives a positive expected number of violations — rather than concluding the method has failed, **build the object anyway and remove the violations**, and if the expected number removed is a small fraction of the whole, the surviving object is still large enough. **It converts a failed argument into a working one surprisingly often.**

3 The Lovász local lemma

LOVÁSZ LOCAL LEMMA: IF BAD EVENTS ARE RARE AND ONLY LOCALLY DEPENDENT, NONE NEED OCCUR

Suppose each bad event has probability at most p , and each depends on at most d of the others. If $e \cdot p \cdot (d+1) \leq 1$, then with positive probability none of the bad events occur.

Which the union bound cannot give: when there are many bad events, the sum of their probabilities exceeds 1 and the union bound says nothing at all (Module 02 §3's wastefulness note). The local lemma is what you use instead.

And the condition depends on the *local* dependency degree rather than the total number of events — so a million bad events, each depending on only ten others, satisfies the condition perfectly well. That independence from the total count is the entire power of the result.

The canonical application is k -SAT: a formula in which each clause shares variables with only a few others is satisfiable, however many clauses it has — which the union bound cannot establish for any formula with more than a handful of clauses. It is also how edge-colouring, frequency-assignment, and scheduling existence results are proved.

AND MOSER–TARDOS MADE IT CONSTRUCTIVE

The local lemma was non-constructive for roughly thirty years — it proved that a satisfying assignment exists and gave no indication of how to find one, which is the usual limitation of the probabilistic method.

Moser and Tardos's algorithm: sample all variables randomly; while some bad event currently holds, resample *only the variables that event depends on*; repeat. It terminates in expected polynomial time under the local lemma's condition. Three lines.

And the proof is an entropy-compression argument — if the algorithm ran for many steps, then a log of which events were resampled would constitute a compression of the random bits consumed to below their entropy, which is impossible. So the algorithm cannot run long.

Which is one of the best results in this subject: a thirty-year-old non-constructive proof made constructive by an algorithm of three lines, with a proof technique nobody had used for the purpose before. And it is worth knowing as an example that non-constructive proofs are sometimes waiting to become algorithms, rather than being permanently different in kind.

4 From existence to algorithm

- MAX-CUT at ratio $1/2$. §1's existence proof *is the algorithm* — flip a coin per vertex (CSCE 669 Module 10 §2).
- MAX-3SAT at $7/8$. A uniformly random assignment satisfies $7/8$ of the clauses in expectation, and $7/8$ is provably optimal (CSCE 637 Module 11 §1) — so the trivial probabilistic algorithm is the best possible, which is both satisfying and deflating.

- **Set cover's logarithmic ratio. Randomised rounding of the linear programming relaxation**, which is Module 08.
- **Moser–Tardos for the local lemma** (§3), which turns the strongest existence tool into a working algorithm.
- **And expander graph and error-correcting code constructions**, where **the probabilistic existence proof came first and explicit constructions followed decades later** (Module 06 §4) — and in the code case the random construction is still what is used. **The pattern is that the expectation argument is usually also the algorithm, and derandomising it is Module 12 §1's subject.**

WHY THIS METHOD IS WORTH LEARNING AS AN ENGINEER

It produces approximation algorithms immediately. 'A random choice achieves the average' gives a **guarantee with no algorithm design at all**, and it is frequently the best known ratio (MAX-3SAT being the clearest case).

It tells you what is achievable before you design anything. The random baseline is a **floor**, and **if your clever algorithm does not beat a coin flip, you have learned something important cheaply.**

And it is the fastest route to a lower bound on what a structure must contain — 'every graph on n vertices has an independent set of size at least X ' — **which is useful in sizing and capacity arguments** well outside combinatorics.

So the habit is: before designing an algorithm for an optimisation problem, compute what a uniformly random choice achieves in expectation. It takes five minutes, it gives you a baseline and sometimes a guarantee, and it is sometimes simply the answer.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Alon & Spencer — The Probabilistic Method	The reference for this module — all the variants of §2 and the local lemma. Library copy; Spencer's free lecture notes cover the core.
Mitzenmacher & Upfal — chapter 6	The method at this course's level , with the MAX-CUT and MAX-3SAT examples worked.
Moser & Tardos — A Constructive Proof of the General Lovász Local Lemma (free)	§3's constructive result — short, and the entropy-compression argument is worth reading in full.
Tao — Moser's entropy compression argument (free blog post)	§3's proof technique explained , more accessibly than the paper.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Prove the MAX-CUT half-the-edges result** yourself, and then implement it and measure the achieved ratio.
- 2 **Prove the MAX-3SAT 7/8 result** by the same argument.
- 3 **Find a problem where linearity suffices and you were about to assume independence**, and note the difference.
- 4 **Apply the deletion method** to a problem where the basic method gives a positive expected number of violations.
- 5 **Apply the alteration method** to prove an independent-set lower bound.
- 6 **Construct a case where the union bound fails** and the local lemma applies.
- 7 **Verify the local lemma's condition** for a k -SAT instance with bounded clause overlap.
- 8 **Implement Moser–Tardos** and measure its resampling count against the bound.
- 9 **For an optimisation problem you care about, compute the random baseline** in expectation.
- 10 **Then beat it, or report that you could not.**

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the probabilistic method and give the MAX-CUT proof.
- 2 Why is that proof also an algorithm?
- 3 Why does linearity of expectation need no independence, and why is it underused?
- 4 Name five variants and when each applies.
- 5 What does the deletion method do?
- 6 State the local lemma and say what the union bound cannot give.
- 7 Why does the local dependency degree matter rather than the event count?
- 8 What did Moser–Tardos achieve, and by what argument?
- 9 Name five probabilistic existence proofs that are algorithms.
- 10 Give the five-minute habit this module recommends.